

Tugas Analisis Multimedia: Image (Citra Digital)

Mata Kuliah: Sistem & Teknologi Multimedia

Nama: Muhammad Fauzi Azizi **NIM:** 122140106

Deskripsi Tugas

Tugas ini bertujuan untuk memahami **representasi dasar data citra digital (image)** melalui praktik langsung memuat data, visualisasi komponen warna, serta melakukan analisis spasial sederhana menggunakan berbagai teknik dasar pengolahan citra.

Anda akan bekerja dengan satu atau beberapa gambar (foto diri, objek, atau lingkungan sekitar) untuk:

- Mengamati struktur data piksel dan channel warna (RGB, Grayscale, HSV, dsb.)
- Menganalisis perbedaan hasil visualisasi antar representasi warna
- Melakukan eksplorasi sederhana terhadap transformasi citra (cropping, filtering, edge detection, dll.)
- Menyimpulkan pengaruh setiap tahap pemrosesan terhadap persepsi visual

Fokus tugas ini adalah pada **pemahaman konsep representasi spasial citra digital** dan **interpretasi hasil visualisasi**, bukan pada manipulasi kompleks atau penerapan model pembelajaran mesin.

Soal 1 — Cropping dan Konversi Warna

- Ambil sebuah gambar diri Anda (*selfie*) menggunakan kamera atau smartphone.
- Lakukan **cropping secara manual** untuk menghasilkan dua potongan:
 - Cropping **kotak persegi** pada area wajah.
 - Cropping **persegi panjang** pada area latar belakang.
- Resize hasil crop menjadi **920×920 piksel**.
- Konversi gambar menjadi **grayscale** dan **HSV**, lalu tampilkan ketiganya berdampingan.
- Tambahkan **anotasi teks** berisi nama Anda di atas kepala pada gambar hasil crop.
 - Gaya teks (font, warna, posisi, ukuran, ketebalan) **dibebaskan**.
- Jelaskan efek **cropping** dan **perubahan warna** menggunakan **Markdown**.

```

In [1]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ===== SETUP =====
BASE_DIR = os.getcwd()          # folder tempat notebook berada      # ganti kal
SELFIE_PATH = os.path.join(BASE_DIR, "foto1.jpg")

print("Base dir :", BASE_DIR)
print("Selfie   :", SELFIE_PATH)

# Fungsi bantu untuk menampilkan gambar BGR di matplotlib
def show_bgr(img, title="", size=(5,5)):
    plt.figure(figsize=size)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
    plt.show()

# ===== LOAD GAMBAR =====
img = cv2.imread(SELFIE_PATH)
if img is None:
    raise FileNotFoundError(f"Tidak bisa membaca gambar: {SELFIE_PATH}")

h, w = img.shape[:2]
print(f"Resolusi asli: {w} x {h}")

show_bgr(img, "Selfie Asli")

# ===== CROPPING =====
# Crop wajah (area tengah)
face_y1 = int(0.25 * h)
face_y2 = int(0.82 * h)
face_x1 = int(0.20 * w)
face_x2 = int(0.80 * w)
face_crop = img[face_y1:face_y2, face_x1:face_x2]

# Crop latar belakang (bagian atas-kanan)
bg_y1 = 0
bg_y2 = int(0.40 * h)
bg_x1 = int(0.40 * w)
bg_x2 = w
bg_crop = img[bg_y1:bg_y2, bg_x1:bg_x2]

show_bgr(face_crop, "Crop Wajah (RGB)")
show_bgr(bg_crop, "Crop Background (RGB)")

# ===== RESIZE 920 x 920 =====
target_size = (920, 920)
face_resized = cv2.resize(face_crop, target_size, interpolation=cv2.INTER_LINEAR)
bg_resized = cv2.resize(bg_crop, target_size, interpolation=cv2.INTER_LINEAR)

show_bgr(face_resized, "Wajah 920x920")
show_bgr(bg_resized, "Background 920x920")

```

```

# ===== KONVERSI GRAYSCALE & HSV =====
face_gray = cv2.cvtColor(face_resized, cv2.COLOR_BGR2GRAY)
face_hsv = cv2.cvtColor(face_resized, cv2.COLOR_BGR2HSV)

# ===== ANOTASI TEKS NAMA =====
face_annotated = face_resized.copy()

nama = "Muhammad Fauzi Azizi" # ganti kalau mau
font = cv2.FONT_HERSHEY_SIMPLEX
scale = 2.0
thickness = 3

text_size, _ = cv2.getTextSize(nama, font, scale, thickness)
text_x = (face_annotated.shape[1] - text_size[0]) // 2
text_y = int(face_annotated.shape[0] * 0.12) # sedikit di bawah tepi atas

cv2.putText(
    face_annotated,
    nama,
    (text_x, text_y),
    font,
    scale,
    (0, 255, 0), # warna hijau
    thickness,
    cv2.LINE_AA
)

# ===== TAMPILKAN 3 REPRESENTASI =====
plt.figure(figsize=(15,5))

# RGB + teks
plt.subplot(1,3,1)
plt.imshow(cv2.cvtColor(face_annotated, cv2.COLOR_BGR2RGB))
plt.title("RGB + Teks")
plt.axis("off")

# Grayscale
plt.subplot(1,3,2)
plt.imshow(face_gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# HSV (ditampilkan sebagai RGB setelah konversi balik)
face_hsv_rgb = cv2.cvtColor(face_hsv, cv2.COLOR_HSV2RGB)
plt.subplot(1,3,3)
plt.imshow(face_hsv_rgb)
plt.title("HSV")
plt.axis("off")

plt.tight_layout()
plt.show()

# ===== SIMPAN HASIL =====
cv2.imwrite(os.path.join(BASE_DIR, "soal1_face_crop.png"), face_resized)
cv2.imwrite(os.path.join(BASE_DIR, "soal1_bg_crop.png"), bg_resized)

```

```
cv2.imwrite(os.path.join(BASE_DIR, "soal1_face_gray.png"), face_gray)
cv2.imwrite(os.path.join(BASE_DIR, "soal1_face_hsv.png"), face_hsv)
cv2.imwrite(os.path.join(BASE_DIR, "soal1_face_annotated.png"), face_annotated)

print("Semua output Soal 1 tersimpan di folder WORKSHEET4.")
```

Base dir : d:\Mulmed\Worksheet.4

Selfie : d:\Mulmed\Worksheet.4\foto1.jpg

Resolusi asli: 1200 x 1600

Selfie Asli



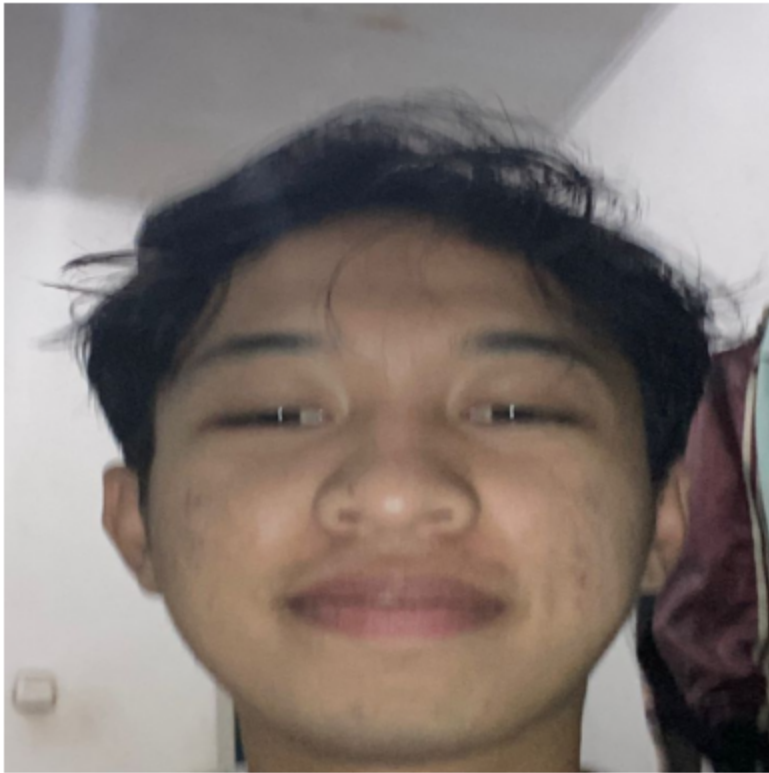
Crop Wajah (RGB)



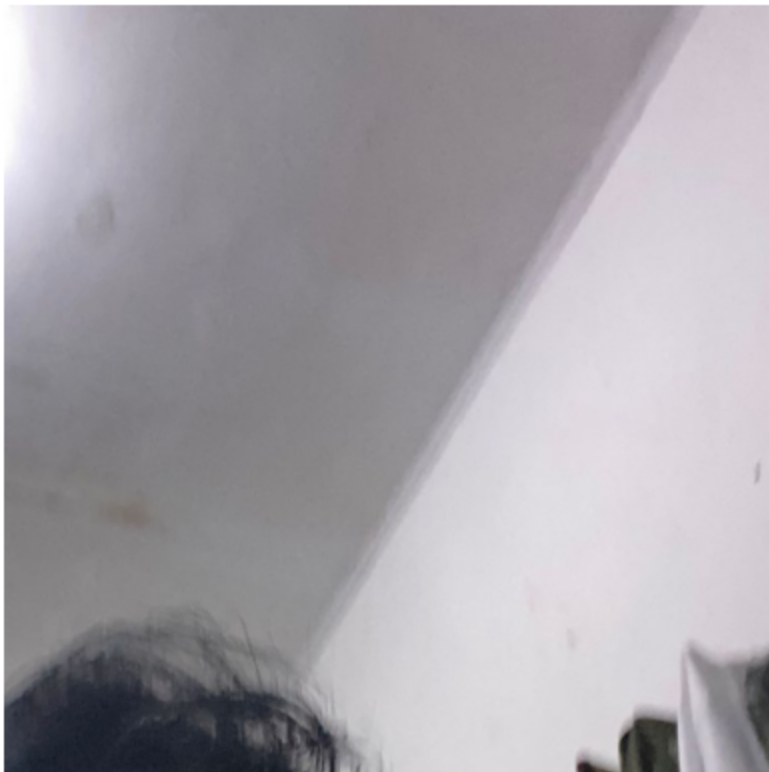
Crop Background (RGB)



Wajah 920x920



Background 920x920





Semua output Soal 1 tersimpan di folder WORKSHEET4.

Soal 2 — Manipulasi Channel Warna RGB

- Gunakan gambar hasil crop dari Soal 1.
- Konversikan gambar ke ruang warna **RGB**.
- Lakukan manipulasi channel warna dengan cara:
 - **Naikkan intensitas channel merah sebanyak 50 poin** (maksimum 255).
 - **Turunkan intensitas channel biru sebanyak 30 poin** (minimum 0).
- Teknik atau cara menaikkan/menurunkan intensitas **dibebaskan**, asalkan logis dan hasilnya terlihat.
- Gabungkan kembali channel warna dan **simpan gambar hasil modifikasi dalam format .png**.
- **Tampilkan histogram per channel (R, G, B)** untuk gambar asli dan hasil modifikasi menggunakan `matplotlib.pyplot.hist`.
- Jelaskan dampak perubahan RGB pada warna gambar dalam sel **Markdown**.

```
In [2]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ==== Pastikan BASE_DIR ada (kalau dari Soal 1 sudah ada, ini cuma overwrite lagi)
BASE_DIR = os.getcwd()
SELFIE_PATH = os.path.join(BASE_DIR, "foto1.jpg")

# ----- Ambil face_resized dari Soal 1, atau buat ulang kalau belum ada -----
try:
    face_resized # cek variabel sudah ada
    print("Pakai face_resized dari Soal 1")
except NameError:
    print("face_resized belum ada, crop ulang dari foto1.jpg")
```

```

img = cv2.imread(SELFIE_PATH)
if img is None:
    raise FileNotFoundError(f"Tidak bisa membaca gambar: {SELFIE_PATH}")
h, w = img.shape[:2]

# Crop area wajah (sama seperti Soal 1)
face_y1 = int(0.25 * h)
face_y2 = int(0.82 * h)
face_x1 = int(0.20 * w)
face_x2 = int(0.80 * w)
face_crop = img[face_y1:face_y2, face_x1:face_x2]

face_resized = cv2.resize(face_crop, (920, 920), interpolation=cv2.INTER_LINEAR)

# ===== KONVERSI BGR → RGB =====
orig_bgr = face_resized.copy()
orig_rgb = cv2.cvtColor(orig_bgr, cv2.COLOR_BGR2RGB)

# Pisah channel
R, G, B = cv2.split(orig_rgb)

# ===== MANIPULASI CHANNEL =====
# Naikkan R +50 (maks 255), turunkan B -30 (min 0)
R_mod = np.clip(R.astype(np.int16) + 50, 0, 255).astype(np.uint8)
B_mod = np.clip(B.astype(np.int16) - 30, 0, 255).astype(np.uint8)
G_mod = G.copy() # tidak diubah

# Gabungkan lagi ke citra RGB
mod_rgb = cv2.merge([R_mod, G_mod, B_mod])
mod_bgr = cv2.cvtColor(mod_rgb, cv2.COLOR_RGB2BGR)

# ===== TAMPILKAN PERBANDINGAN =====
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(orig_rgb)
plt.title("Wajah Asli (RGB)")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(mod_rgb)
plt.title("Wajah Modifikasi (R+50, B-30)")
plt.axis("off")

plt.tight_layout()
plt.show()

# ===== HISTOGRAM PER CHANNEL =====
fig, axes = plt.subplots(2, 3, figsize=(15,8))
channels_orig = [R, G, B]
channels_mod = [R_mod, G_mod, B_mod]
titles = ["R", "G", "B"]

for i in range(3):
    # Asli
    axes[0, i].hist(channels_orig[i].ravel(), bins=256, range=(0,255))

```

```

axes[0, i].set_title(f"Histogram {titles[i]} Asli")
axes[0, i].set_xlim(0, 255)

# Modifikasi
axes[1, i].hist(channels_mod[i].ravel(), bins=256, range=(0,255))
axes[1, i].set_title(f"Histogram {titles[i]} Modifikasi")
axes[1, i].set_xlim(0, 255)

plt.tight_layout()
plt.show()

# ===== SIMPAN HASIL =====
cv2.imwrite(os.path.join(BASE_DIR, "soal2_face_rgb_original.png"), orig_bgr)
cv2.imwrite(os.path.join(BASE_DIR, "soal2_face_rgb_modified.png"), mod_bgr)

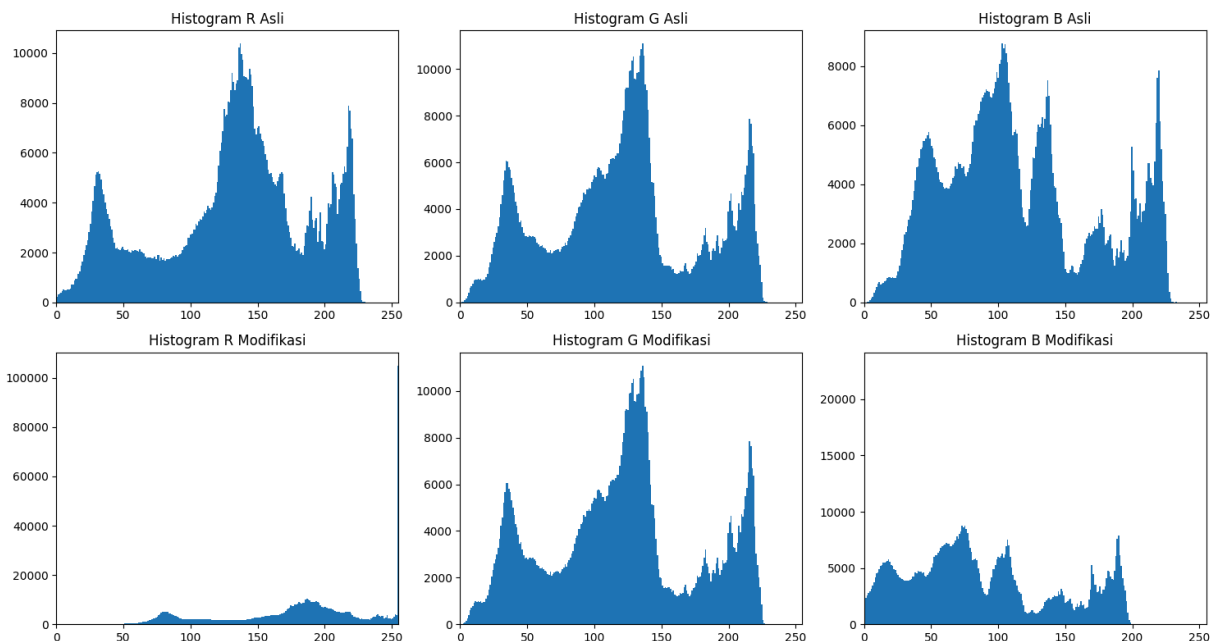
```

Pakai face_resized dari Soal 1

Wajah Asli (RGB)



Wajah Modifikasi (R+50, B-30)



Out[2]: True

Penjelasan dari gambar : Perubahan nilai channel RGB pada gambar memberikan dampak yang signifikan terhadap tampilan akhir citra. Pada percobaan ini, channel merah (R) dinaikkan sebesar +50, sementara channel biru (B) diturunkan sebesar -30, dan channel hijau (G) dibiarkan tetap. Peningkatan intensitas channel merah menyebabkan keseluruhan gambar menjadi lebih hangat; warna kulit tampak lebih cerah dan kemerahan, serta area terang di ruangan berubah menjadi sedikit kekuningan.

Soal 3 — Deteksi Tepi dan Filter Citra

- Ambil gambar **objek dengan background bertekstur** (misalnya kain bermotif, jerami, atau batu).
- Terapkan **edge detection (Canny)** dan tampilkan hasilnya.
- Lakukan **thresholding dengan nilai ambang tertentu** (bebas Anda tentukan) agar hanya objek utama yang tersisa.
- Buat **bounding box** di sekitar objek hasil segmentasi (boleh manual atau otomatis).
- Terapkan **filter blur** dan **filter sharpening**, lalu **bandingkan hasil keduanya**.
- Jelaskan bagaimana setiap filter memengaruhi detail gambar dalam format **Markdown**.

```
In [3]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR = os.getcwd()
IMG_PATH = os.path.join(BASE_DIR, "foto3.png")

print("Path gambar:", IMG_PATH)

# ===== 1. Load Gambar Objek =====
img = cv2.imread(IMG_PATH)
if img is None:
    raise FileNotFoundError(f"Gambar tidak ditemukan: {IMG_PATH}")

h, w = img.shape[:2]
print(f"Resolusi gambar: {w} x {h}")

# Fungsi bantu untuk tampil BGR di matplotlib
def show_bgr(img, title="", size=(5,5)):
    plt.figure(figsize=size)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
    plt.show()

show_bgr(img, "Gambar Asli")

# ===== 2. Konversi ke Grayscale =====
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.figure(figsize=(5,5))
plt.imshow(gray, cmap="gray")
```

```

plt.title("Grayscale")
plt.axis("off")
plt.show()

# ===== 3. Edge Detection (Canny) =====
edges = cv2.Canny(gray, threshold1=80, threshold2=180)

plt.figure(figsize=(5,5))
plt.imshow(edges, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")
plt.show()

# ===== 4. Thresholding untuk Segmentasi Objek Utama =====
# Kamu boleh ganti nilai 150 kalau terlalu terang/gelap
_, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(5,5))
plt.imshow(thresh, cmap="gray")
plt.title("Threshold Biner")
plt.axis("off")
plt.show()

# ===== 5. Cari Kontur & Buat Bounding Box =====
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

bbox_img = img.copy()

if len(contours) == 0:
    print("Tidak ada kontur terdeteksi, coba ubah nilai threshold.")
else:
    # Ambil kontur dengan area terbesar sebagai 'objek utama'
    largest = max(contours, key=cv2.contourArea)
    x, y, w_bb, h_bb = cv2.boundingRect(largest)
    cv2.rectangle(bbox_img, (x, y), (x + w_bb, y + h_bb), (0, 0, 255), 3)

    show_bgr(bbox_img, "Bounding Box Objek Utama")

# ===== 6. Filter Blur (Gaussian Blur) =====
blurred = cv2.GaussianBlur(img, (9, 9), 0)
show_bgr(blurred, "Gaussian Blur")

# ===== 7. Filter Sharpening =====
sharpen_kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]])
sharpened = cv2.filter2D(img, ddepth=-1, kernel=sharpen_kernel.astype(np.float32))
show_bgr(sharpened, "Sharpened Image")

# ===== 8. Bandingkan Blur vs Sharpen =====
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(blurred, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Blur")
plt.axis("off")

```

```
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(sharpened, cv2.COLOR_BGR2RGB))
plt.title("Sharpened")
plt.axis("off")

plt.tight_layout()
plt.show()

# ===== 9. Simpan Hasil ke File =====
cv2.imwrite(os.path.join(BASE_DIR, "soal3_gray.png"), gray)
cv2.imwrite(os.path.join(BASE_DIR, "soal3_edges.png"), edges)
cv2.imwrite(os.path.join(BASE_DIR, "soal3_thresh.png"), thresh)
cv2.imwrite(os.path.join(BASE_DIR, "soal3_bbox.png"), bbox_img)
cv2.imwrite(os.path.join(BASE_DIR, "soal3_blur.png"), blurred)
cv2.imwrite(os.path.join(BASE_DIR, "soal3_sharpen.png"), sharpened)
```

Path gambar: d:\Mulmed\Worksheet.4\foto3.png

Resolusi gambar: 910 x 607

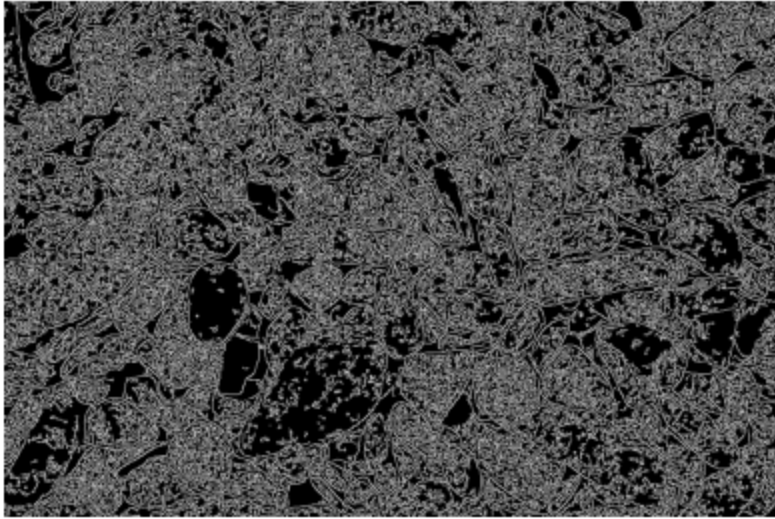
Gambar Asli



Grayscale



Canny Edge Detection



Threshold Biner



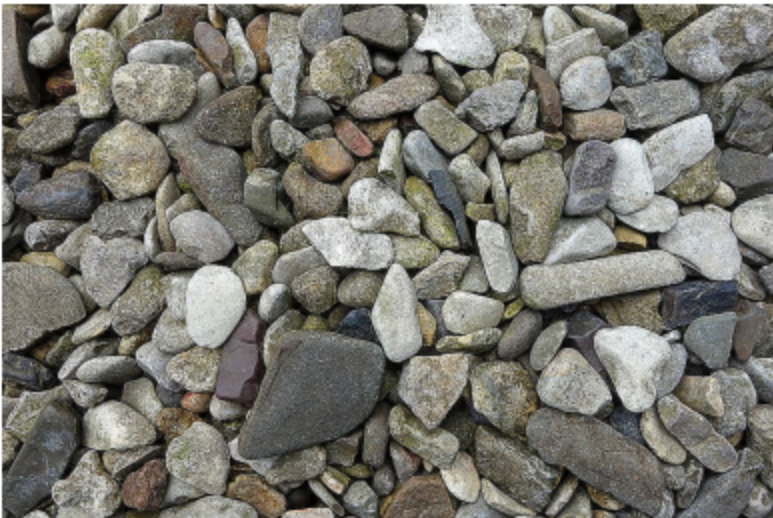
Bounding Box Objek Utama



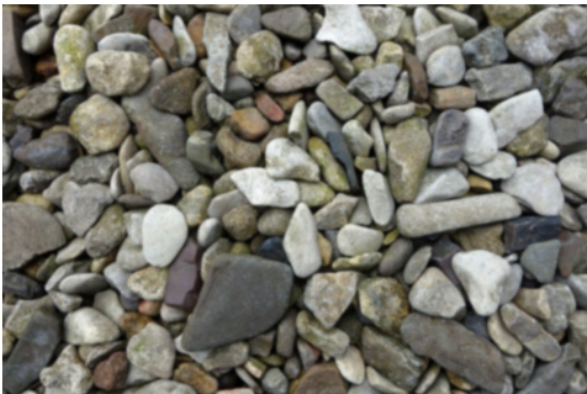
Gaussian Blur



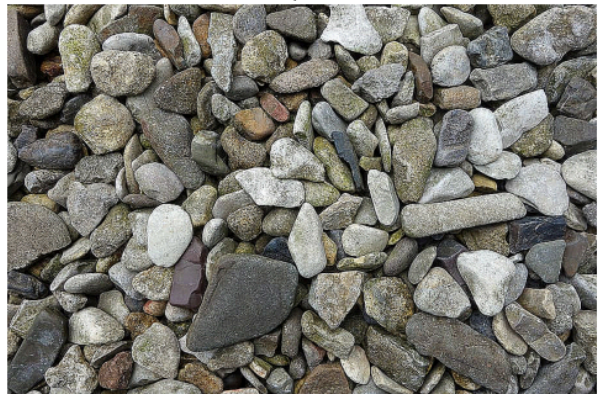
Sharpened Image



Gaussian Blur



Sharpened



Out[3]: True

Penjelasan pada gambar : Filter citra memberikan efek berbeda terhadap detail visual gambar. Gaussian Blur bekerja dengan meratakan nilai piksel menggunakan kernel Gaussian sehingga tekstur halus dan noise menjadi berkurang. Akibatnya, citra tampak lebih

lembut dan detail kecil menghilang, termasuk sebagian kontur objek. Sebaliknya, Sharpening meningkatkan perbedaan intensitas antar piksel, terutama di area tepi. Hal ini membuat garis batas objek lebih tegas dan detail tekstur lebih menonjol. Namun, sharpening juga dapat membuat noise ikut terlihat lebih jelas. Dengan demikian, blur mengurangi detail untuk menghasilkan citra yang lebih halus, sedangkan sharpening memperkuat detail untuk meningkatkan ketajaman visual gambar.

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

- Ambil gambar diri Anda dengan ekspresi wajah **netral**.
- Lakukan **deteksi wajah dan landmark** menggunakan salah satu dari:
 - **MediaPipe**, atau
 - **Dlib**, atau
 - **OpenCV**.
- Buat **overlay filter digital kreatif** karya Anda sendiri, misalnya:
 - topi, kumis, masker, helm, aksesoris, atau bentuk unik lainnya.
 - Filter boleh dibuat dari **gambar eksternal (PNG)** atau digambar langsung (misal bentuk lingkaran, garis, poligon, dll).
- Pastikan posisi overlay menyesuaikan **landmark wajah** dengan logis.
- **Gunakan alpha blending sebagai saran** agar hasil tampak lebih natural.
- Tampilkan perbandingan antara **gambar asli** dan **hasil dengan filter**.
- Jelaskan bagaimana Anda menghitung posisi overlay dan tantangan yang dihadapi selama implementasi (gunakan **Markdown**).

```
In [4]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import mediapipe as mp

# ===== SETUP =====
BASE_DIR = os.getcwd()
IMG_PATH = os.path.join(BASE_DIR, "foto4.jpg") # ganti jika ingin pakai foto selfi

print("Load gambar:", IMG_PATH)

img = cv2.imread(IMG_PATH)
if img is None:
    raise FileNotFoundError(f"Tidak bisa membaca gambar: {IMG_PATH}")

h, w = img.shape[:2]

def show_bgr(image, title=""):
```

```

plt.figure(figsize=(5,5))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title(title)
plt.axis("off")
plt.show()

# ===== MEDIA PIPE FACE MESH =====
mp_face_mesh = mp.solutions.face_mesh

with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:

    # BGR → RGB, pastikan format cocok dengan MediaPipe
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_rgb = np.ascontiguousarray(img_rgb, dtype=np.uint8)

    results = face_mesh.process(img_rgb)

    if not results.multi_face_landmarks:
        print("❌ Wajah tidak terdeteksi")
        filtered = img.copy()
    else:
        landmarks = results.multi_face_landmarks[0].landmark

        # ===== AMBIL KOORDINAT MATA DARI LANDMARK =====
        # Indeks penting:
        # - 33, 133 : mata kanan (dari sudut pandang kamera)
        # - 362, 263 : mata kiri
        idx_right_outer = 33
        idx_right_inner = 133
        idx_left_outer = 362
        idx_left_inner = 263

        def lm_to_xy(idx):
            lm = landmarks[idx]
            return int(lm.x * w), int(lm.y * h)

        rx_o, ry_o = lm_to_xy(idx_right_outer)
        rx_i, ry_i = lm_to_xy(idx_right_inner)
        lx_o, ly_o = lm_to_xy(idx_left_outer)
        lx_i, ly_i = lm_to_xy(idx_left_inner)

        # ===== HITUNG FRAME KACAMATA =====
        # Lebar tiap lensa = jarak horizontal antara inner & outer eye + margin
        right_eye_width = abs(rx_i - rx_o)
        left_eye_width = abs(lx_i - lx_o)

        # Gunakan tinggi lensa proporsional terhadap lebar
        lens_height_r = int(right_eye_width * 0.8)
        lens_height_l = int(left_eye_width * 0.8)

        # Posisi vertikal kira-kira di tengah antara dua titik mata

```

```

ry_center = int((ry_o + ry_i) / 2)
ly_center = int((ly_o + ly_i) / 2)

# Tambah margin
margin = 10

# Bounding box Lensa kanan
r_x1 = min(rx_o, rx_i) - margin
r_x2 = max(rx_o, rx_i) + margin
r_y1 = ry_center - lens_height_r // 2
r_y2 = ry_center + lens_height_r // 2

# Bounding box Lensa kiri
l_x1 = min(lx_o, lx_i) - margin
l_x2 = max(lx_o, lx_i) + margin
l_y1 = ly_center - lens_height_l // 2
l_y2 = ly_center + lens_height_l // 2

# ===== BUAT OVERLAY KACAMATA =====
overlay = img.copy()

color = (0, 0, 0) # hitam (frame kaca mata)
thickness = 6

# Gambar bingkai Lensa kanan & kiri
cv2.rectangle(overlay, (r_x1, r_y1), (r_x2, r_y2), color, thickness)
cv2.rectangle(overlay, (l_x1, l_y1), (l_x2, l_y2), color, thickness)

# Gambar jembatan kaca mata (bridge) di antara dua Lensa
bridge_y = int((ry_center + ly_center) / 2)
bridge_x1 = r_x2
bridge_x2 = l_x1
cv2.line(overlay, (bridge_x1, bridge_y), (bridge_x2, bridge_y), color, thic

# Gambar sedikit gagang kaca mata ke arah samping
# kanan
cv2.line(overlay, (r_x1, ry_center), (r_x1 - 40, ry_center - 10), color, th
# kiri
cv2.line(overlay, (l_x2, ly_center), (l_x2 + 40, ly_center - 10), color, th

# ===== ALPHA BLENDING =====
alpha = 0.8
filtered = cv2.addWeighted(overlay, alpha, img, 1 - alpha, 0)

# ===== TAMPILKAN HASIL =====
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(filtered, cv2.COLOR_BGR2RGB))
plt.title("Dengan Filter Kacamata")
plt.axis("off")

```

```
plt.tight_layout()
plt.show()

# ===== SIMPAN =====
out_path = os.path.join(BASE_DIR, "soal4_filter_kacamata.png")
cv2.imwrite(out_path, filtered)
```

Load gambar: d:\Mulmed\Worksheet.4\foto4.jpg

d:\Mulmed\Worksheet.4\.venv\lib\site-packages\google\protobuf\symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use message_factory.getMessageClass() instead. SymbolDatabase.GetPrototype() will be removed soon.
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '

Gambar Asli



Dengan Filter Kacamata



Out[4]: True

Penjelasan : Untuk menentukan posisi overlay kacamata, pertama saya menggunakan landmark mata dari MediaPipe FaceMesh. MediaPipe memberikan koordinat ter-normalisasi (0–1), sehingga setiap titik saya ubah menjadi koordinat piksel dengan mengalikan nilai x dengan lebar gambar (w) dan nilai y dengan tinggi gambar (h). Dua pasang landmark penting digunakan untuk setiap mata: sudut luar dan sudut dalam mata kanan (indeks 33 dan 133) serta mata kiri (indeks 362 dan 263). Dari selisih koordinat horizontal kedua titik tersebut, saya memperoleh lebar mata, yang kemudian digunakan untuk menentukan ukuran lensa kacamata. Tinggi lensa dibuat proporsional terhadap lebar mata, sedangkan posisi vertikal diperoleh dari rata-rata posisi kedua landmark sehingga frame kacamata berada tepat di sekitar area mata secara alami. Setelah itu, saya membuat dua bounding box untuk masing-masing lensa dan menambahkan garis penghubung di tengah sebagai

jembatan kacamata serta sedikit garis samping sebagai gagangnya. Seluruh bentuk digambar di atas layer overlay dan kemudian digabungkan kembali ke gambar asli menggunakan alpha blending agar tampak natural.

Tantangan utama dalam implementasi ini adalah memastikan ukuran kacamata tetap proporsional untuk berbagai bentuk dan orientasi wajah. Landmark yang bergeser sedikit saja akan memengaruhi posisi dan ukuran lensa, terutama jika wajah miring atau tidak menghadap tegak ke kamera. Selain itu, MediaPipe membutuhkan format array tertentu (uint8 dan memori kontigu), sehingga konversi RGB harus dilakukan dengan benar agar tidak muncul error tipe. Tantangan lainnya adalah mengatur margin, ketebalan garis, dan nilai alpha yang tepat agar kacamata terlihat jelas namun tetap menyatu dengan gambar asli tanpa menutupi detail mata secara berlebihan.

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

- Ambil gambar **objek datar** seperti karya tangan di kertas, tulisan di papan tulis, atau foto produk di meja dengan kondisi pencahayaan atau sudut yang tidak ideal.
- Lakukan **preprocessing** untuk memperbaiki tampilan agar lebih rapi dan jelas, dengan langkah-langkah:
 - Konversi ke **grayscale**.
 - **Koreksi perspektif (transformasi homografi)** menggunakan **4 titik manual** agar objek terlihat sejajar dan tidak terdistorsi.
 - Terapkan **thresholding adaptif atau Otsu** (pilih salah satu, dan jelaskan alasan pilihan Anda).
- Tampilkan **setiap tahap pemrosesan dalam satu grid** agar mudah dibandingkan.
- Jelaskan fungsi masing-masing tahap dan bagaimana teknik ini meningkatkan kualitas visual citra (gunakan **Markdown**).

```
In [5]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ===== SETUP =====
BASE_DIR = os.getcwd()
IMG_PATH = os.path.join(BASE_DIR, "foto5.png") # ganti jika ingin pakai foto lain

print("Load gambar:", IMG_PATH)

img = cv2.imread(IMG_PATH)
if img is None:
    raise FileNotFoundError(f"Tidak bisa membaca gambar: {IMG_PATH}")

h, w = img.shape[:2]
```

```

def show(img, title="", cmap=None):
    plt.figure(figsize=(5,5))
    if cmap:
        plt.imshow(img, cmap=cmap)
    else:
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
    plt.show()

# ===== 1. GRAYSCALE =====
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
show(gray, "Grayscale", cmap="gray")

# ===== 2. KOREKSI PERSPEKTIF (HOMOGRAFI) =====
# Tentukan 4 titik sudut kertas secara MANUAL (urutan: kiri atas, kanan atas, kanan
# Sesuaikan angka di bawah dengan kondisi gambar kamu
src_pts = np.float32([
    [250, 180], # kiri atas
    [820, 200], # kanan atas
    [860, 1000], # kanan bawah
    [230, 980]  # kiri bawah
])

# Ukuran output hasil pelurusan kertas
width_out = 600
height_out = 900

dst_pts = np.float32([
    [0, 0],
    [width_out, 0],
    [width_out, height_out],
    [0, height_out]
])

M = cv2.getPerspectiveTransform(src_pts, dst_pts)
warp = cv2.warpPerspective(gray, M, (width_out, height_out))
show(warp, "Setelah Koreksi Perspektif", cmap="gray")

# ===== 3. THRESHOLDING (OTSU) =====
_, otsu = cv2.threshold(warp, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
show(otsu, "Threshold Otsu", cmap="gray")

# ===== 4. GRID VISUALISASI =====
plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(2,2,2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

```

```
plt.subplot(2,2,3)
plt.imshow(warp, cmap="gray")
plt.title("Setelah Koreksi Perspektif")
plt.axis("off")

plt.subplot(2,2,4)
plt.imshow(otsu, cmap="gray")
plt.title("Threshold Otsu")
plt.axis("off")

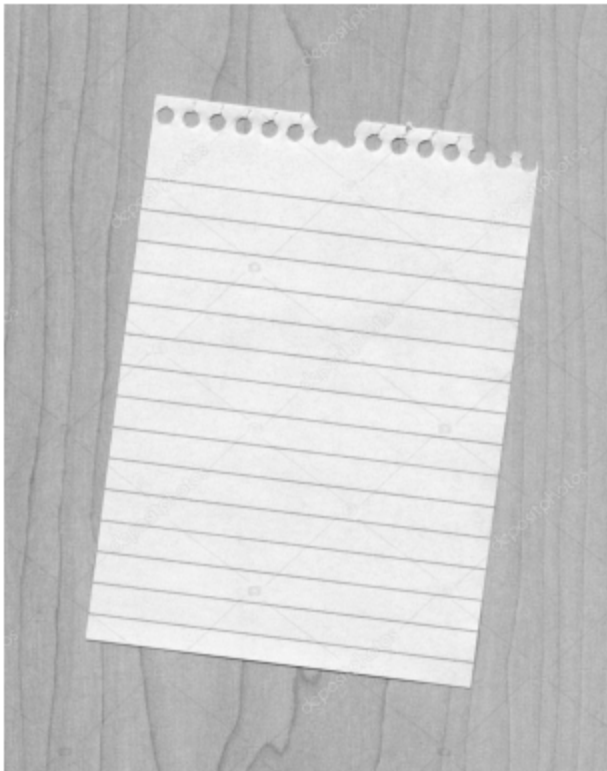
plt.tight_layout()
plt.show()

# ===== SIMPAN FILE =====
cv2.imwrite(os.path.join(BASE_DIR, "soal5_gray.png"), gray)
cv2.imwrite(os.path.join(BASE_DIR, "soal5_warp.png"), warp)
cv2.imwrite(os.path.join(BASE_DIR, "soal5_otsu.png"), otsu)

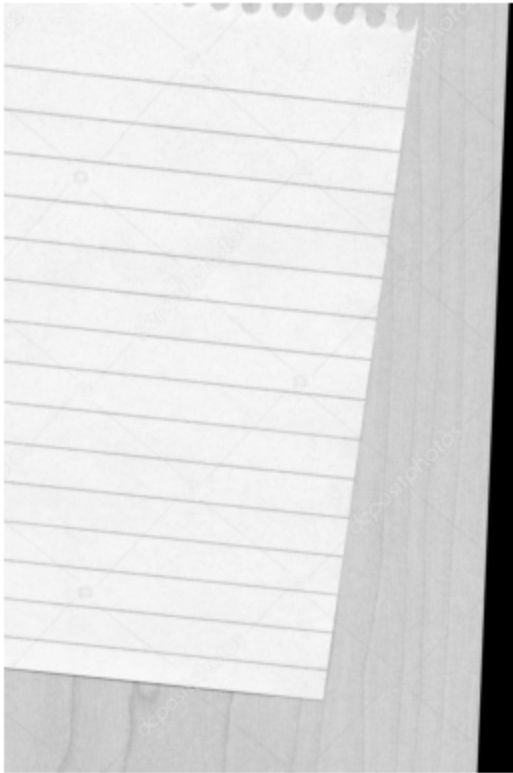
print("Soal 5 selesai! Semua hasil telah disimpan.")
```

Load gambar: d:\Mulmed\Worksheet.4\foto5.png

Grayscale



Setelah Koreksi Perspektif



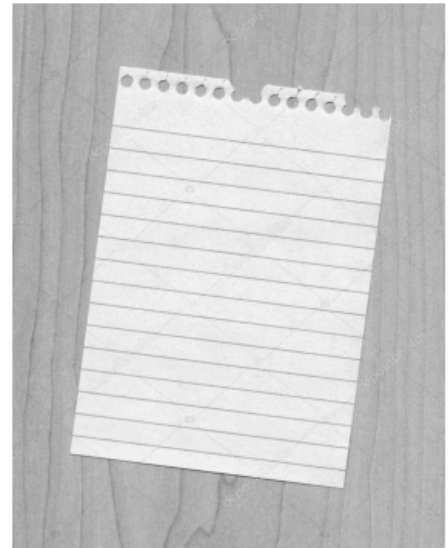
Threshold Otsu



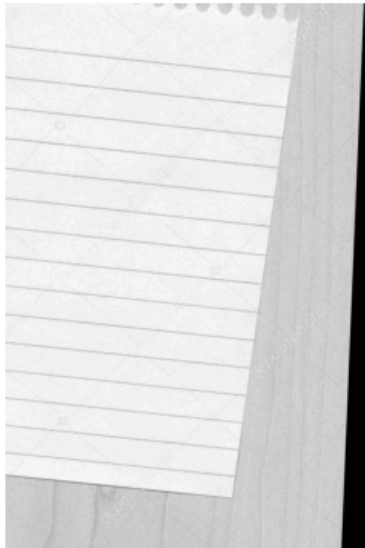
Gambar Asli



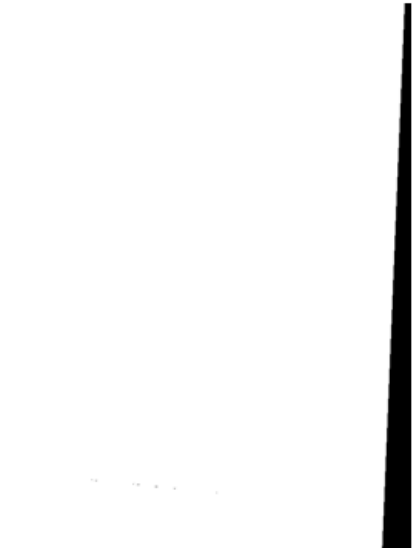
Grayscale



Setelah Koreksi Perspektif



Threshold Otsu



Soal 5 selesai! Semua hasil telah disimpan.

Pada soal ini saya melakukan tiga tahap utama untuk meningkatkan kualitas citra dokumen.

1. Konversi ke Grayscale

Langkah ini menyederhanakan citra menjadi intensitas terang-gelap sehingga proses segmentasi menjadi lebih mudah. Warna kayu dan warna kertas tidak lagi relevan sehingga grayscale memberikan representasi yang lebih stabil.

2. Koreksi Perspektif (Homografi)

Karena foto diambil pada sudut miring, bentuk kertas tampak seperti trapesium. Empat titik sudut kertas dipilih secara manual dan dipetakan ke empat titik tujuan yang membentuk persegi panjang sempurna. Matriks homografi dihitung menggunakan `cv2.getPerspectiveTransform`, kemudian citra diproyeksikan ulang menggunakan `cv2.warpPerspective` sehingga kertas tampak lurus seperti hasil scanner.

3. Thresholding Otsu

Metode Otsu digunakan untuk memisahkan foreground (tulisan) dan background (kertas) secara otomatis dengan mencari nilai ambang terbaik berdasarkan distribusi intensitas piksel. Hasilnya adalah citra biner dengan tulisan yang kontras dan mudah dibaca.

Ketiga langkah ini menghasilkan citra dokumen yang lebih rapi, sejajar, dan lebih mudah digunakan untuk kebutuhan pembacaan, pengarsipan, maupun analisis lebih lanjut.

Aturan Umum Pengerjaan

- Kerjakan secara **mandiri**.
- Bantuan AI (seperti ChatGPT, Copilot, dsb.) diperbolehkan **dengan bukti percakapan** ([screenshot](#) / [link](#) / [script percakapan](#)).
- Source code antar mahasiswa harus berbeda.
- Jika mendapat bantuan teman, tuliskan nama dan NIM teman yang membantu.
- Plagiarisme akan dikenakan sanksi sesuai aturan akademik ITERA.
- Cantumkan seluruh **credit dan referensi** yang digunakan di bagian akhir notebook.
- Penjelasan setiap soal ditulis dalam **Markdown**, bukan di dalam komentar kode.

Aturan Pengumpulan

- Semua file kerja Anda (notebook `.ipynb`, gambar, dan hasil) **wajib diunggah ke GitHub repository tugas sebelumnya**.

- Gunakan struktur folder berikut di dalam repo Anda:

```
/Nama_NIM_Repo/ # Nama repo sebelumnya
├── assets_ws4/   # berisi semua gambar atau video asli (input)
├── results_ws4/  # berisi semua hasil modifikasi dan output
├── worksheet4.ipynb
└── NIM_Worksheet4.pdf
```

- File yang dikumpulkan ke **Tally** hanya berupa **hasil PDF** dari notebook Anda, dengan format nama:

NIM_Worksheet4.pdf
- Pastikan notebook telah dijalankan penuh sebelum diekspor ke PDF.
- Sertakan tautan ke repository GitHub Anda di bagian atas notebook atau di halaman pertama PDF.

✓ Catatan Akhir

Worksheet 4 ini bertujuan mengasah pemahaman Anda tentang manipulasi citra digital secara praktis. Gunakan kreativitas Anda untuk menghasilkan hasil

visual yang menarik dan penjelasan konseptual yang jelas.